

FRAUD DETECTION



Rédigé par

LACROIX Ewan
ABOUBAKAR Ali Ibrahim
DIALLO Ousmane Djounnou

Table des matières

1	Introduction	2
2	Analyse descriptive	3
2.1	Analyse des variables brutes (Amount et Time)	3
2.2	Sélection des variables discriminantes (test de Kruskal-Wallis)	4
2.3	Analyse de la multicolinéarité (VIF)	4
2.4	Détection et traitement des valeurs extrêmes	5
3	Méthodes d'apprentissage non supervisées	5
4	Méthodes d'apprentissage supervisées	6
4.1	Présentation des modèles	6
4.1.1	Régression logistique	6
4.1.2	Random Forest	7
4.1.3	AdaBoost	7
4.1.4	XGBoost	7
4.1.5	Réseau de neurones	8
4.2	Mesures de performance et critères de comparaison	8
4.2.1	Recall	8
4.2.2	Precision	8
4.2.3	Accuracy	8
4.2.4	G-Mean	8
4.2.5	AUC-ROC.	9
4.2.6	AUC-PR.	9
4.2.7	Cross-Entropy	9
4.2.8	Score de Brier.	9
4.3	Séparation de la base	9
4.4	Résultats sans rééchantillonnage	10
4.5	Méthodes de traitement des données déséquilibrées	10
4.6	Résultats avec Rééchantillonnage	11
5	Approche par minimisation des coûts	12
6	Conclusion	15
7	Bibliographie	16
8	Annexes	17

1 Introduction

De nos jours, la lutte contre la fraude constitue un enjeu majeur en raison de ses conséquences financières et sociales croissantes. Cette problématique est d'autant plus préoccupante que le nombre de victimes d'escroqueries et de fraudes aux moyens de paiement ne cesse d'augmenter. En France, les services de police et de gendarmerie nationales ont recensé 411 700 victimes en 2023 contre 417 300 en 2024, soit une hausse de 1,4 %, témoignant d'une progression certes ralentie mais toujours persistante¹. Cette évolution s'explique notamment par la diversification des techniques employées par les fraudeurs, un phénomène renforcé par l'essor récent de technologies telles que l'intelligence artificielle.

Parmi les différentes formes de fraude, la fraude à la carte bancaire figure aujourd'hui parmi les plus répandues, en particulier avec l'essor des transactions en ligne observé depuis la crise sanitaire liée au Covid-19. Elle correspond à l'utilisation non autorisée d'une carte de paiement, ou d'un moyen de paiement assimilé, dans le but d'obtenir frauduleusement de l'argent ou des biens. En France, ce type de fraude a représenté en 2024 un montant total de 519 millions d'euros de pertes déclarées², illustrant l'ampleur économique de ce phénomène.

L'un des défis majeurs de la détection de la fraude réside dans son caractère à la fois rare et évolutif. Les transactions frauduleuses ne constituent qu'une infime proportion du volume total des opérations, entraînant un fort déséquilibre des classes dans les jeux de données. Dans ce contexte, les approches classiques de machine learning tendent à favoriser la classe majoritaire des transactions légitimes, au détriment de la détection d'événements frauduleux pourtant critiques. Par ailleurs, les schémas de fraude évoluent rapidement afin de contourner les dispositifs de sécurité existants, ce qui impose le recours à des modèles capables de s'adapter en continu. Cette double contrainte limite l'efficacité des méthodes d'apprentissage automatique traditionnelles lorsqu'aucune stratégie spécifique de rééquilibrage des données ou d'adaptation dynamique n'est mise en œuvre.

Dans ce rapport, nous commençons par réaliser une analyse descriptive des données afin d'identifier les principales caractéristiques des transactions frauduleuses. Nous mettons ensuite en œuvre des techniques d'apprentissage non supervisé pour la détection de transactions suspectes ou d'anomalies. Dans une deuxième partie, nous évaluons les performances de plusieurs méthodes d'apprentissage supervisé. Une troisième partie est consacrée à l'introduction de différentes méthodes de rééchantillonnage, suivie d'une nouvelle implémentation des modèles supervisés. Enfin, nous comparons et analysons l'ensemble des résultats obtenus afin d'évaluer l'impact de ces différentes approches sur la détection de la fraude.

1. Ministère de l'Intérieur, « *Insécurité et délinquance en 2024* », 9^e édition du bilan statistique.

2. Observatoire de la sécurité des moyens de paiement, *Rapport annuel*, 2024.

2 Analyse descriptive

La base de données recense **284 807 transactions** effectuées par des titulaires de cartes européens sur une période de 48 heures, en septembre 2013. Ce jeu de données structuré est complet et ne présente aucune valeur manquante.

Pour représenter ces opérations, nous disposons majoritairement de variables issues d'une transformation PCA. Seules deux variables brutes ont été conservées :

Time : délai en secondes écoulé depuis la première transaction de l'historique.

Amount : montant de la transaction en euros.

Quant à la variable cible **Class**, elle agit comme un indicateur binaire où 0 désigne les transactions normales et 1 les fraudes.

Cependant, la répartition des données révèle un **fort déséquilibre**. Avec seulement 492 cas de fraude, la classe minoritaire ne représente que 0,172 % du volume total. Cette disproportion rend les indicateurs classiques, tels que l'accuracy, peu pertinents. Elle impose dès lors l'utilisation de métriques spécifiques et de techniques de rééquilibrage pour éviter de biaiser l'analyse.

2.1 Analyse des variables brutes (Amount et Time)

L'étude de ces deux variables permet d'identifier des différences de comportement entre les transactions légitimes et les fraudes. Le tableau ci-dessous résume les statistiques principales :

TABLE 1 – Statistiques descriptives

Variable	Moyenne	Médiane	Min	Max
Amount (€)	88,35	22,00	0,00	25 691,16
Time (heures)	26,34	23,53	0,00	48,00

La répartition des montants montre un très grand écart entre la moyenne (88,35 €) et la médiane (22,00 €). Cela indique que la majorité des transactions concerne de petites sommes, mais que la moyenne est augmentée par quelques achats très élevés, certains dépassant 25 000 €.

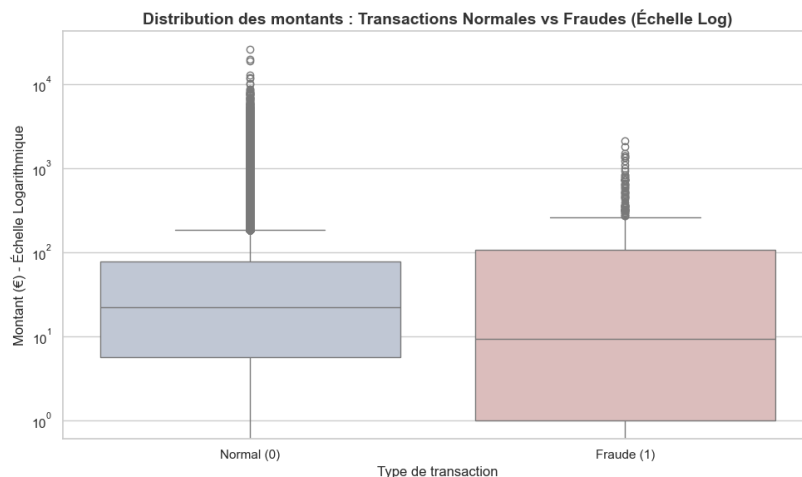


FIGURE 1 – Distribution des montants : Transactions Normales vs Fraudes (Échelle Log)

Comparaison Normal vs Fraude Les transactions frauduleuses présentent une distribution très asymétrique : le montant moyen (122 €) est largement supérieur à la médiane (9 €). Cela indique que la majorité des fraudes concernent de petits montants, avec seulement quelques valeurs élevées.

À l'inverse, les transactions normales sont plus régulières autour de leur médiane (22 €), même si elles comportent aussi de gros montants. Par conséquent, le montant seul ne suffit pas à distinguer clairement les transactions frauduleuses des transactions normales.

La variable **Time** permet d'observer l'activité sur deux jours complets. Pour l'analyse, il est préférable de transformer cette variable afin de considérer non pas le temps écoulé, mais l'heure précise de la transaction (cycle de 24h).

Cette transformation montre que les transactions classiques suivent un rythme habituel : activité faible la nuit et élevée en journée. Le profil de la fraude est cependant très différent.

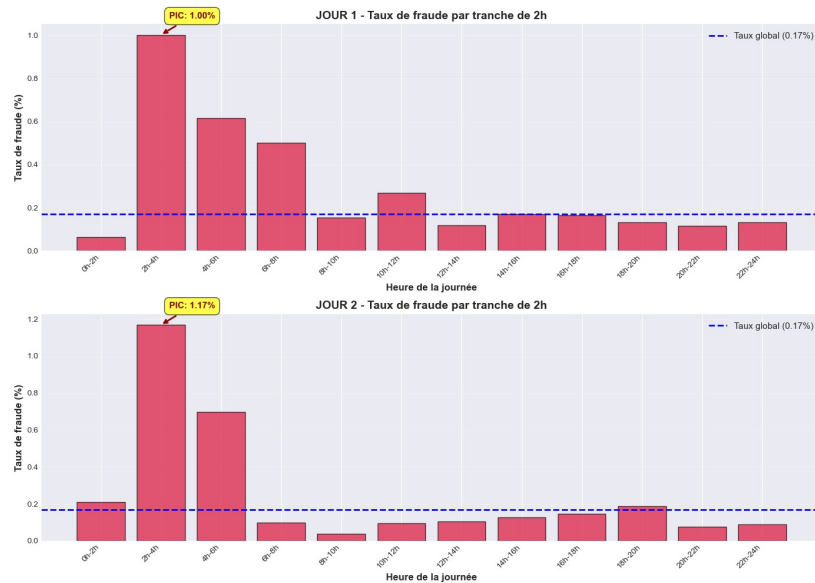


FIGURE 2 – Taux de fraude par tranche horaire sur 2 jours

Comme illustré ci-dessus, le taux de fraude atteint un maximum critique entre 2h et 4h du matin. Il grimpe à 1,00 % la première nuit et 1,17 % la seconde, soit bien au-dessus de la moyenne globale (0,17 %).

Ce schéma se répète à l'identique d'un jour à l'autre, mettant en évidence un effet horaire marqué : les attaques semblent ciblées sur les créneaux où la vigilance des titulaires est minimale (sommeil), retardant ainsi l'opposition de la carte.

2.2 Sélection des variables discriminantes (test de Kruskal-Wallis)

Afin d'évaluer la capacité des variables à distinguer les transactions frauduleuses des transactions normales, des tests de Kruskal-Wallis ont été réalisés. Ce test non paramétrique permet de comparer la distribution des variables explicatives entre les deux groupes.

Les résultats montrent que certaines variables présentent des différences statistiquement significatives entre les transactions frauduleuses et normales ($p\text{-value} < 0,05$), tandis que d'autres ne montrent pas de différence significative entre les deux groupes.

TABLE 2 – Résultats du test de Kruskal-Wallis sur les variables sélectionnées

Variable	Statistique (K-W)	p-value
V14	1189.04	< 0.0001
V4	1132.02	< 0.0001
V12	1125.74	< 0.0001
...	...	...
V15	2.30	0.1295
V22	1.24	0.2664

2.3 Analyse de la multicolinéarité (VIF)

Pour évaluer les liens entre les variables, nous avons privilégié le coefficient de corrélation de Spearman, plus adapté que celui de Pearson lorsque les données ne suivent pas une loi normale.

L'analyse des corrélations met en évidence une relation entre la variable **Amount** et la composante **V2**, avec un coefficient de Spearman $r_s = 0.50$. Cette dépendance suggère une redondance partielle d'information.

Pour quantifier précisément cet impact, nous utilisons le Facteur d'Inflation de la Variance (VIF) :

$$VIF_j = \frac{1}{1 - R_j^2} \quad (0)$$

Les résultats confirment cette observation :

- Les variables **V1 à V28** présentent des VIF faibles (inférieurs au seuil usuel de 5), validant l'indépendance des composantes principales.
- La variable **Amount**, en revanche, affiche un VIF élevé ($>$ de 10), corrélation directe avec V2.

Malgré ce VIF élevé, nous conservons la variable **Amount** en raison de son pouvoir discriminant. De plus, les modèles d'ensemble (comme Random Forest ou XGBoost) gèrent bien ces corrélations, et des techniques de **régularisation** pourront être appliquées si nous utilisons des modèles linéaires.

2.4 Détection et traitement des valeurs extrêmes

Nous analysons la présence de valeurs extrêmes susceptibles d'affecter la performance des modèles de détection de fraude.

Approche univariée (IQR) : L'analyse par boîtes à moustaches révèle que la variable **Amount** présente 11.19% de valeurs extrêmes, confirmant la forte variabilité des montants de transactions. Les composantes principales issues de la PCA présentent généralement des taux d'outliers faibles à modérés.

Approches multivariées : Nous avons comparé plusieurs méthodes de détection multivariée incluant Isolation Forest, Local Outlier Factor (LOF), One-Class SVM, Elliptic Envelope et DBSCAN. Ces algorithmes détectent entre 10% et 15% d'outliers dans le dataset, révélant la présence de comportements atypiques dans les données.

Le taux d'outliers détecté dépassant largement le seuil usuel de 2% qui justifierait une suppression, nous avons opté pour une stratégie de conservation. Nous procédons à une discrétisation des variables par arbre de décision (méthode CART). Cette transformation permet de réduire l'impact des valeurs extrêmes tout en préservant le pouvoir discriminant de ces variables pour la modélisation.

3 Méthodes d'apprentissage non supervisées

La détection de fraude repose généralement sur des jeux de données étiquetés, dans lesquels chaque transaction est classée comme frauduleuse ou légitime. Toutefois, dans la réalité, ces étiquettes peuvent être non fiables. En particulier, il existe un risque de double mauvaise spécification (double misspecification), qui affecte à la fois les données et les modèles utilisés pour l'apprentissage.

D'une part, la mauvaise spécification des données provient du fait que certaines transactions frauduleuses peuvent être incorrectement étiquetées comme non frauduleuses. Cela s'explique notamment par la nature adaptative de la fraude : les fraudeurs cherchent activement à dissimuler leurs comportements en imitant les caractéristiques des transactions légitimes afin d'échapper aux systèmes de détection. En conséquence, une partie des fraudes reste non détectée lors des processus d'annotation, ce qui introduit un bruit dans les labels. Ce type d'erreur est particulièrement problématique car les faux négatifs (fraudes labellisées comme non fraudes) peuvent être beaucoup plus fréquents que l'inverse.

D'autre part, la mauvaise spécification du modèle résulte de l'hypothèse selon laquelle les labels fournis sont corrects et que les distributions des classes sont bien séparables dans l'espace des caractéristiques. Or, lorsque des fraudes sont volontairement déguisées pour ressembler à des transactions normales, cette hypothèse est violée. Dans ce contexte, un modèle peut apprendre à considérer certains schémas frauduleux comme normaux, réduisant ainsi sa capacité à détecter des fraudes sophistiquées ou émergentes. Ce phénomène peut également entraîner une dégradation des performances hors échantillon, en particulier lorsque les stratégies de fraude évoluent dans le temps.

Face à cette double problématique, les méthodes d'apprentissage non supervisé apparaissent comme une alternative pertinente, ou du moins comme un complément aux approches supervisées. En effet, ces méthodes ne reposent pas sur les étiquettes de classe lors de la phase d'apprentissage, ce qui les rend, par construction, moins sensibles aux erreurs de labellisation. Plutôt que d'apprendre une séparation explicite entre fraude et

non-fraude, elles cherchent à modéliser la structure des données et à identifier des observations atypiques susceptibles de correspondre à des comportements frauduleux.

Ainsi, dans cette section nous avons appliqué deux algorithmes non supervisés : K-means, Isolation Forest et HDBSCAN, afin d'identifier des transactions suspectes ou anormales et d'évaluer leur capacité à détecter des fraudes.

Comme le montre le tableau suivant, environ 25,14% des transactions identifiées comme anormales par Isolation Forest correspondent effectivement à des fraudes, tandis que près de 84,15% des observations considérées comme du bruit par HDBSCAN sont des transactions frauduleuses.

TABLE 3 – Matrices de confusion des méthodes non supervisées

HDBSCAN			Isolation Forest		
Réalité	Prédiction		Réalité	Prédiction	
	Normal	Anomalie		Normal	Anomalie
Normal	93 830	190 485	Normal	283 955	360
Fraude	78	414	Fraude	367	125

4 Méthodes d'apprentissage supervisées

L'apprentissage supervisé regroupe un ensemble de méthodes statistiques et algorithmiques visant à modéliser la relation entre une variable d'intérêt (la cible) et un ensemble de variables explicatives (les caractéristiques), à partir d'observations étiquetées. Dans le cadre de la détection de fraude, la cible est typiquement binaire, indiquant si une transaction est frauduleuse ($y = 1$) ou légitime ($y = 0$).

On note $\{(x_i, y_i)\}_{i=1}^n$ un échantillon d'apprentissage, où $x_i \in R^p$ est le vecteur de caractéristiques de la transaction i et $y_i \in \{0, 1\}$. L'objectif consiste à apprendre une fonction de décision $f(x)$ produisant soit une probabilité $\hat{p}(x) = P(y = 1 | x)$, soit un score ensuite transformé en décision via un seuil τ , par exemple $1\{\hat{p}(x) \geq \tau\}$.

Dans cette section, cinq modèles supervisés sont mobilisés et comparés : la régression logistique, la forêt aléatoire (Random Forest), AdaBoost, XGBoost et un réseau de neurones.

4.1 Présentation des modèles

4.1.1 Régression logistique

La régression logistique est un modèle probabiliste paramétrique largement utilisé en économétrie pour la classification binaire. Elle suppose que la probabilité conditionnelle de fraude s'écrit

$$P(y = 1 | x) = \sigma(x^\top \beta) = \frac{1}{1 + \exp(-x^\top \beta)},$$

où $\beta \in R^p$ est un vecteur de paramètres et $\sigma(\cdot)$ est la fonction logistique.

L'estimation est effectuée par maximisation de la log-vraisemblance (équivalente à la minimisation de la perte logistique). Pour un échantillon $\{(x_i, y_i)\}$, la perte (négative) s'écrit

$$\mathcal{L}(\beta) = - \sum_{i=1}^n \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right], \quad \text{où } \hat{p}_i = \sigma(x_i^\top \beta).$$

La régression logistique offre une interprétabilité directe, ses limites en fraude proviennent principalement de sa structure linéaire (dans l'espace des caractéristiques) et de sa difficulté à capturer des interactions complexes sans ingénierie de variables (termes d'interaction, transformations non linéaires). Elle constitue néanmoins une référence solide, notamment pour la calibration et comme modèle benchmark.

4.1.2 Random Forest

Une Random Forest est un ensemble d'arbres de décision entraînés selon une stratégie de bagging. Le procédé est le suivant :

- Tirer B échantillons bootstrap de l'échantillon d'apprentissage.
- Entraîner un arbre de décision sur chaque échantillon.
- À chaque nœud de l'arbre, sélectionner aléatoirement un sous-ensemble de variables (de taille m) et ne considérer que ces variables pour le meilleur découpage.

La prédiction finale agrège les arbres : moyenne des probabilités $\hat{p}_b(x)$ ou vote majoritaire. Pour la probabilité :

$$\hat{p}(x) = \frac{1}{B} \sum_{b=1}^B \hat{p}_b(x).$$

Les Random Forests capturent naturellement des non-linéarités et interactions et réduisent la variance par agrégation. Elles gèrent bien des relations complexes sans spécification paramétrique explicite. En contrepartie, la taille des modèles peut être importante, l'interprétation globale est moins directe qu'une régression logistique.

4.1.3 AdaBoost

AdaBoost est une méthode de boosting qui combine séquentiellement des classifieurs faibles (souvent des arbres très peu profonds). L'idée consiste à accorder progressivement plus de poids aux observations mal classées afin de concentrer l'apprentissage sur les cas difficiles.

Schématiquement, on maintient une distribution de poids $\{w_i^{(t)}\}$ sur les observations. À chaque itération t , un classifieur faible $h_t(x)$ est entraîné pour minimiser l'erreur pondérée, puis reçoit un poids α_t croissant lorsque sa performance est bonne. La prédiction finale est une somme pondérée :

$$F(x) = \sum_{t=1}^T \alpha_t h_t(x), \quad \hat{y} = 1\{F(x) \geq 0\},$$

et une sortie probabiliste peut être obtenue via une transformation (selon l'implémentation).

AdaBoost peut être performant lorsque des règles simples combinées suffisent à séparer fraude et non-fraude, et il est souvent compétitif avec un bon réglage. Toutefois, il peut être sensible au bruit et aux outliers : des observations aberrantes peuvent être sur-pondérées au fil des itérations, ce qui dégrade la généralisation si ces points sont des erreurs de mesure plutôt que des fraudes informatives.

4.1.4 XGBoost

XGBoost appartient à la famille des méthodes de gradient boosting à arbres. Il construit un modèle additif de la forme

$$\hat{y}(x) = \sum_{t=1}^T f_t(x), \quad f_t \in \mathcal{F},$$

où chaque f_t est un arbre de décision. À l'itération t , on ajoute un arbre qui corrige les erreurs résiduelles du modèle courant, en minimisant une fonction objectif composée d'une perte de prédiction et d'un terme de régularisation :

$$\mathcal{J} = \sum_{i=1}^n \ell(y_i, \hat{y}_i) + \sum_{t=1}^T \Omega(f_t).$$

La régularisation $\Omega(\cdot)$ pénalise notamment la complexité des arbres (nombre de feuilles, poids des feuilles), ce qui limite le sur-apprentissage.

XGBoost intègre également des mécanismes de taux d'apprentissage, de sous-échantillonnage, et une optimisation efficace pour grands volumes de données.

Dans des problèmes fortement non linéaires et déséquilibrés comme la fraude, XGBoost est souvent un utilisé grâce à son excellent compromis biais-variance, sa capacité à modéliser des interactions. Ses limites principales concernent la sensibilité aux hyperparamètres et la nécessité d'une validation rigoureuse.

4.1.5 Réseau de neurones

Un réseau de neurones de type perceptron multicouche (MLP) approxime une fonction non linéaire par composition de couches. Pour une couche cachée, on peut écrire :

$$h(x) = \phi(W_1x + b_1), \quad \hat{p}(x) = \sigma(W_2h(x) + b_2),$$

où W_1, W_2 sont des matrices de poids, b_1, b_2 des biais, ϕ une activation et σ la sigmoïde pour produire une probabilité.

L'entraînement consiste à minimiser une perte (souvent l'entropie croisée) par descente de gradient stochastique et rétropropagation. Des techniques de régularisation sont nécessaires pour éviter le sur-apprentissage : pénalisation L_2 , arrêt précoce, éventuellement dropout.

Les réseaux de neurones sont attractifs lorsque la relation entre caractéristiques et fraude est complexe, avec de nombreuses interactions et effets non linéaires. En revanche, sur des données tabulaires classiques, ils exigent un réglage plus délicat (architecture, taux d'apprentissage, normalisation, régularisation). L'interprétabilité est également plus faible.

4.2 Mesures de performance et critères de comparaison

Après optimisation des hyperparamètres pour chacun de nos modèles et calculs sur échantillon test, nous devons calculer des mesures de performances afin de les comparer. Nous avons choisi les mesures suivantes :

4.2.1 Recall

Le recall mesure la capacité du modèle à détecter les fraudes effectivement présentes :

$$\text{Recall} = \frac{TP}{TP + FN}.$$

Un recall élevé limite les fraudes manquées, ce qui est généralement central lorsque le coût d'un faux négatif est élevé.

4.2.2 Precision

La precision mesure la fiabilité des alertes émises par le modèle :

$$\text{Precision} = \frac{TP}{TP + FP}.$$

Une precision élevée implique peu de fausses alertes parmi les cas signalés, ce qui est déterminant si l'investigation humaine est coûteuse ou si une action automatique (blocage) est déclenchée.

4.2.3 Accuracy

L'accuracy est la proportion globale de prédictions correctes :

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

Dans un contexte très déséquilibré (fraude rare), l'accuracy peut être peu informative, car prédire systématiquement la classe majoritaire peut déjà donner une accuracy élevée.

4.2.4 G-Mean

Le G-Mean cherche un compromis entre la performance sur la classe minoritaire (fraude) et la classe majoritaire (non-fraude). On définit la spécificité

$$\text{Specificity} = \frac{TN}{TN + FP},$$

puis

$$\text{G-Mean} = \sqrt{\text{Recall} \times \text{Specificity}}.$$

Cette métrique pénalise les solutions extrêmes (par exemple, un modèle qui détecte tout en générant trop de faux positifs, ou l'inverse).

4.2.5 AUC-ROC.

La courbe ROC trace $TPR = \text{Recall}$ en fonction de $FPR = \frac{FP}{FP+TN}$ pour tous les seuils. L'aire sous cette courbe (AUC-ROC) mesure la capacité de classement du modèle, et s'interprète comme la probabilité qu'un exemple positif reçoive un score supérieur à un exemple négatif tirés au hasard :

$$\text{AUC-ROC} = P(s(X^+) > s(X^-)).$$

L'AUC-ROC est utile pour comparer des modèles sur leur pouvoir discriminant global, mais peut sembler élevée même lorsque la précision opérationnelle reste faible dans des bases extrêmement déséquilibrées.

4.2.6 AUC-PR.

La courbe Precision-Recall trace la précision en fonction du recall pour tous les seuils. L'aire sous cette courbe (AUC-PR) est souvent plus pertinente lorsque la classe positive est rare, car la précision est directement affectée par la proportion de faux positifs parmi les alertes. Une référence utile est le niveau de base (baseline) égal à la prévalence $\pi = P(y = 1)$: un classifieur aléatoire a une précision attendue proche de π , quel que soit le recall.

4.2.7 Cross-Entropy

La cross-entropy (entropie croisée) évalue la qualité probabiliste via la vraisemblance, en pénalisant fortement les erreurs « confiantes ». Pour une sortie $\hat{p}_i \in (0, 1)$:

$$\text{CE} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)].$$

Plus la cross-entropy est faible, meilleure est la qualité des probabilités au sens de la loss logarithmique.

4.2.8 Score de Brier.

Le score de Brier mesure l'erreur quadratique moyenne des probabilités prédites :

$$\text{Brier} = \frac{1}{n} \sum_{i=1}^n (\hat{p}_i - y_i)^2.$$

Un score de Brier faible indique des probabilités proches des fréquences observées, et donc une meilleure calibration (bien que la calibration puisse être étudiée plus finement via des courbes de calibration).

4.3 Séparation de la base

L'échantillon initial, composé de 284 807 observations, est scindé en deux sous-échantillons disjoints selon un découpage stratifié par la variable de fraude (classe positive), afin de préserver la même proportion de transactions frauduleuses dans chaque sous-ensemble. Plus précisément, 70% des données sont allouées à l'entraînement et 30% sont réservées à un échantillon test, lequel est conservé exclusivement pour l'évaluation finale hors-échantillon.

La sélection des hyperparamètres est réalisée uniquement au sein de la partie entraînement (70%), elle-même subdivisée en deux sous-parties, également de manière stratifiée par fraude : 75% pour l'ajustement des modèles et 25% pour un échantillon de validation (ou calibration) destiné à comparer les configurations d'hyperparamètres selon un schéma hold-out interne. La comparaison des cinq modèles est ensuite effectuée sur la base de leurs performances mesurées sur l'échantillon test (30%) uniquement, l'échantillon de validation n'étant mobilisé que pour l'optimisation des hyperparamètres.

4.4 Résultats sans rééchantillonnage

Le tableau suivant regroupe les résultats de nos différents modèles évalués sur l'échantillon test de la base initiale :

TABLE 4 – Tableau comparatif des modèles de détection de fraude.

Modèle	Recall	Precision	AUC ROC	AUC PR	Accuracy	G Mean	Cross Entropy	Brier Score	F1 Score
LogisticRegression	0.770 270 3	0.826 087 0	0.975 878 2	0.753 066 5	0.999 321 2	0.877 526 9	0.102 215 5	0.021 304 0	0.797 202 8
RandomForest	0.790 540 5	0.921 259 8	0.959 513 7	0.836 718 4	0.999 520 1	0.889 071 3	0.006 592 4	0.000 471 2	0.850 909 1
NeuralNet	0.750 000 0	0.932 773 1	0.985 042 9	0.832 121 7	0.999 473 3	0.865 984 8	0.002 866 4	0.000 490 6	0.831 460 7
AdaBoost	0.797 297 3	0.797 297 3	0.970 957 8	0.713 417 4	0.999 297 8	0.892 758 0	0.169 694 9	0.028 006 4	0.797 297 3
XGBoost	0.790 540 5	0.921 259 8	0.977 409 7	0.812 555 5	0.999 520 1	0.889 071 3	0.007 439 5	0.001 169 6	0.850 909 1

En utilisant la base initiale sans resampling, notre meilleur modèle est le Random Forest. Nous privilégions ici l'aire sous la courbe precision recall et donc aussi le f1 score. Nous remarquons tout de même que les autres modèles bénéficient d'une meilleur AUC mais ce qui nous intéresse ici en fraude ce sont le recall et la précision et au vu du coût dans cette base faible des fraudes, nous devons aussi se doter d'une précision satisfaisante pour éviter de trop gros coûts d'investigation. Le Random Forest remplit ces conditions cependant le XGBoost le suit avec un meilleur score de Brier mais un peu derrière en AUC PR et meilleur en AUC ROC. Rappelons aussi que pour ces critères, le seuil choisit est celui qui maximisait le F1-score.

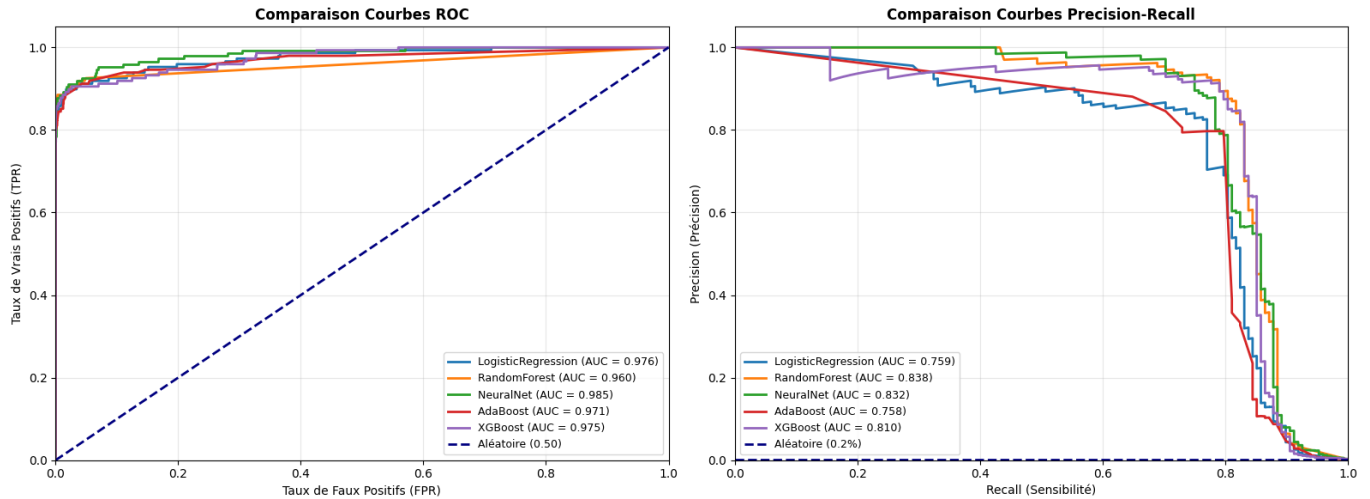


FIGURE 3 – Comparaison des courbes AUC ROC et PR | Base Initiale

Les modèles sont presque similaires ici et le choix dépend de l'arbitrage effectué plus haut, en globalité, ici il serait plus judicieux sur base Initiale de garder le XGBoost ou le Random forest.

4.5 Méthodes de traitement des données déséquilibrées

Le déséquilibre entre classes constitue un problème majeur en détection de fraude. Dans notre jeu de données, les transactions frauduleuses ne représentent que 1,7% des observations. Cette forte asymétrie peut biaiser l'apprentissage des modèles de classification.

Afin de corriger ce déséquilibre, nous avons appliqué plusieurs méthodes de rééchantillonnage selon trois ratios cibles : 5%/95%, 10%/90% et 20%/80% (classe minoritaire / classe majoritaire). Nous avons retenu trois grandes familles de méthodes de rééchantillonnage : le sur-échantillonnage, le sous-échantillonnage et les méthodes hybrides. Ces approches visent respectivement à augmenter le nombre d'observations de la classe minoritaire, à réduire celui de la classe majoritaire, ou à combiner les deux stratégies.

Les méthodes de sur-échantillonnage consistent à augmenter le nombre d'observations de la classe minoritaire afin de rééquilibrer la distribution des classes. Dans cette catégorie, nous avons utilisé les techniques suivantes :

- **Random Over-Sampling (ROS)** : Cette méthode consiste à dupliquer aléatoirement des observations de la classe minoritaire jusqu'à atteindre le ratio souhaité. Elle est simple à mettre en œuvre, mais peut entraîner un sur-apprentissage en raison de la répétition exacte des mêmes observations [?].

- **Synthetic Minority Over-sampling Technique (SMOTE)** : La méthode SMOTE [Chawla et al., 2002] génère de nouvelles observations synthétiques à partir des exemples existants de la classe minoritaire. Pour chaque observation, l'un de ses k plus proches voisins est sélectionné et un nouvel exemple est créé par interpolation linéaire. Cette approche permet une meilleure couverture de l'espace des caractéristiques de la classe minoritaire [Tremblay et al., 2022].
- **Adaptive Synthetic Sampling (ADASYN)** : ADASYN [He et al., 2008] est une extension de SMOTE qui génère davantage d'observations synthétiques dans les régions où la classe minoritaire est faiblement représentée. Le sur-échantillonnage est ainsi adapté à la difficulté locale du problème, ce qui améliore la modélisation des zones complexes.
- **Borderline-SMOTE** : Cette variante de SMOTE se concentre sur les observations minoritaires situées à proximité de la frontière de décision. Ces observations, entourées majoritairement d'exemples de la classe majoritaire, sont considérées comme particulièrement informatives pour améliorer la capacité de discrimination du modèle [?].
- **SVM-SMOTE** : SVM-SMOTE utilise une frontière de décision estimée à l'aide d'un classifieur SVM pour identifier les observations minoritaires proches de la marge. Des exemples synthétiques sont ensuite générés autour de ces points afin de renforcer la séparation entre les deux classes [?].

Les méthodes de sous-échantillonnage visent à réduire le nombre d'observations de la classe majoritaire afin de rééquilibrer la distribution des classes. Dans notre étude, nous avons utilisé les techniques suivantes :

- **Random Under-Sampling (RUS)** : Cette méthode consiste à supprimer aléatoirement des observations de la classe majoritaire jusqu'à atteindre le ratio souhaité. Bien qu'elle permette de réduire le temps de calcul, elle peut entraîner une perte d'informations si des observations pertinentes sont éliminées.
- **NearMiss** : NearMiss sélectionne les observations de la classe majoritaire en fonction de leur distance à la classe minoritaire. Nous utilisons la version 1, qui conserve les observations majoritaires les plus proches des exemples minoritaires. Cette stratégie permet de préserver les observations les plus pertinentes pour la frontière de décision [?].

Les méthodes hybrides quant à elles combinent des techniques de sur-échantillonnage et de sous-échantillonnage afin de tirer parti des avantages des deux approches. Dans ce cadre, nous avons utilisé les méthodes suivantes :

- **SMOTE-ENN** : Cette méthode combine le sur-échantillonnage SMOTE et le nettoyage des données par Edited Nearest Neighbors (ENN). Après la génération d'exemples synthétiques, les observations dont la classe diffère de celle de la majorité de leurs k plus proches voisins sont supprimées. Cette approche permet d'obtenir une frontière de décision plus robuste [Batista et al., 2004].
- **Condensed Nearest Neighbor (CNN)** : La méthode CNN [Gowda and Krishna, 1979] vise à réduire la taille de l'ensemble d'entraînement tout en conservant l'information discriminante. L'algorithme initialise l'ensemble avec toutes les observations de la classe minoritaire, puis ajoute uniquement les observations majoritaires nécessaires à une classification correcte. On obtient ainsi un sous-ensemble plus compact et représentatif.

Pour chaque méthode de rééchantillonnage, nous avons généré trois ensembles d'entraînement correspondant aux ratios 5%/95%, 10%/90% et 20%/80%, à l'exception de la méthode CNN qui ne dépend pas d'un ratio prédéfini. Au total, 25 ensembles d'entraînement ont été construits (8 méthodes $\times$ 3 ratios + CNN). L'ensemble de test est conservé inchangé et déséquilibré afin d'évaluer les performances des modèles dans des conditions proches d'un contexte réel.

4.6 Résultats avec Rééchantillonnage

Nous avons répertorié tous les résultats dans des tableaux en annexe nous permettant de comparer l'ensemble de nos modèles sur toutes nos bases soit au total 130 modèles. Nous remarquons que peu importe les bases, les résultats de la régression logistique pénalisée sont toujours plus faibles que pour les autres. Les Random Forest et XGboost se retrouvent en première position comme nous pouvions nous y attendre au vu de leur structure.

Notons que le rééchantillonnage a porté ses fruits, nous arrivons à obtenir de meilleurs résultats notamment pour les XGboost et via les méthodes SMOTE pour l'ensemble. C'est en particulier la méthode classique SMOTE échantillonnée en 10/90 qui nous offre les meilleurs résultats via le modèle XGBoost. Retrouvons les résultats des modèles entraînés sur cette base dans le tableau suivant :

TABLE 5 – Tableau comparatif des modèles pour la configuration SMOTE 10/90.

Modèle	Recall	Precision	AUC ROC	AUC PR	Accuracy	G Mean	Cross Entropy	Brier Score	F1 Score
AdaBoost	0.756 756 8	0.794 326 2	0.975 078 5	0.722 831 2	0.999 239 3	0.869 769 8	0.191 329 6	0.035 054 8	0.775 086 5
LogisticRegression	0.777 027 0	0.839 416 1	0.974 977 0	0.749 008 0	0.999 356 3	0.881 377 7	0.049 747 2	0.008 464 8	0.807 017 5
NeuralNet	0.722 973 0	0.930 434 8	0.950 320 1	0.796 111 7	0.999 426 5	0.850 238 3	0.009 179 8	0.000 613 7	0.813 688 2
RandomForest	0.790 540 5	0.886 363 6	0.977 747 0	0.841 736 8	0.999 461 6	0.889 045 3	0.009 271 7	0.001 046 3	0.835 714 3
XGBoost	0.810 810 8	0.937 500 0	0.967 424 4	0.852 445 2	0.999 578 7	0.900 408 1	0.003 811 6	0.000 499 0	0.869 565 2

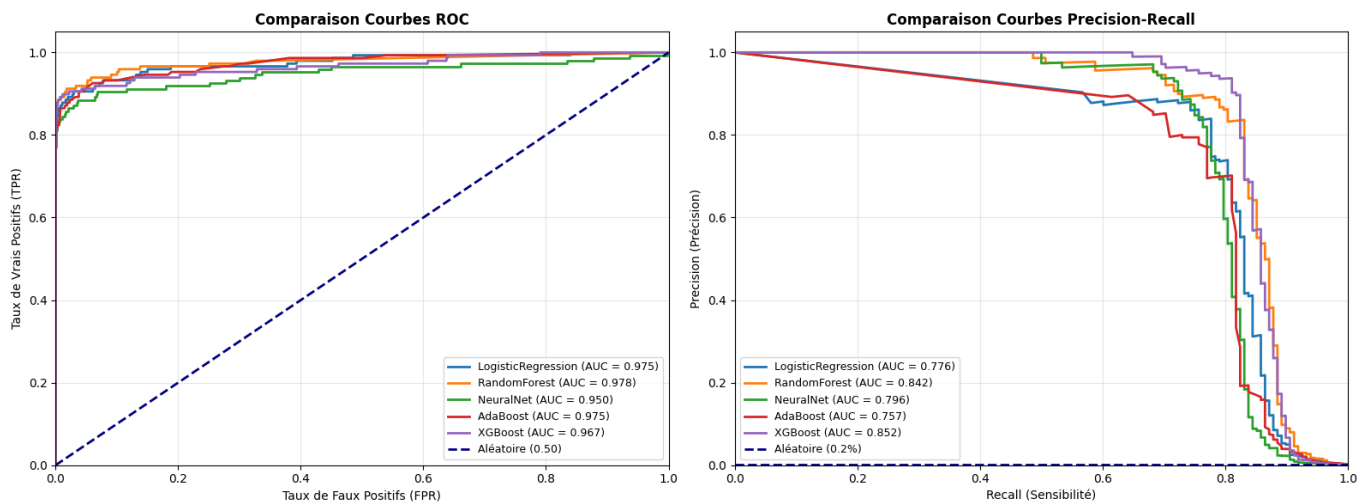


FIGURE 4 – Comparaison des courbes AUC ROC et PR | SMOTE 10/90

Le XGBoost obtient vraiment des résultats remarquables en comparaison à la logistique et l'ADABOOST.

Nous préconisons dans ce contexte d'utiliser donc le modèle XGBoost entraîné sur SMOTE 10/90 pour la détection de fraude, il bénéficie d'un recall de 81% et d'une précision de 93,75%. Au vu du montant moyen des fraudes qui est assez faible environ 120 euros, l'arbitrage coût d'investigation et coût de la fraude doit être fait et donc le F1-score est important et cela se remarque bien sur sa matrice de confusion en annexe.

5 Approche par minimisation des coûts

Les méthodes de classification que nous avons utilisé jusque là, font une hypothèse implicite : toutes les erreurs de classification ont le même coût. En d'autres termes, un faux positif (transaction légitime classée comme frauduleuse) et un faux négatif (transaction frauduleuse classée comme légitime) sont considérés comme équivalents lors de l'optimisation du modèle. Cette hypothèse se manifeste clairement dans la fonction de perte utilisée par la régression logistique classique, qui attribue le même poids à toutes les observations, indépendamment des conséquences financières réelles d'une erreur de classification.

Dans le contexte de la fraude, cette hypothèse peut être inadaptée à la réalité. En effet, les coûts associés aux différents types d'erreurs ne sont pas les mêmes. D'une part, un faux positif engendre principalement des coûts administratifs et d'investigation : l'équipe de surveillance doit examiner manuellement la transaction signalée, contacter potentiellement le client, et consacrer du temps et des ressources à la vérification. D'autre part, un faux négatif entraîne une perte financière directe correspondant au montant de la transaction frauduleuse.

De plus, les bénéfices d'une classification correcte diffèrent également. Un vrai négatif (transaction légitime correctement classée) ne génère aucun coût supplémentaire. En revanche, un vrai positif engendre certes un coût d'investigation, mais permet d'éviter une perte financière potentiellement importante. Cette structure de coûts justifie pleinement le recours à une approche sensible aux coûts.

Pour formaliser cette problématique, nous définissons une matrice de coûts $C_i(\hat{y}|y)$ qui représente le coût associé à la prédiction de la classe $\hat{y}$ lorsque la vraie classe est y , pour la transaction i . Cette matrice se structure comme suit :

	Prédit fraude ($\hat{y} = 1$)	Prédit légitime ($\hat{y} = 0$)
Vraie fraude ($y = 1$)	$C_i(1 1) = c_f$ (Vrai positif)	$C_i(0 1) = A_i$ (Faux négatif)
Vraie légitime ($y = 0$)	$C_i(1 0) = c_f$ (Faux positif)	$C_i(0 0) = 0$ (Vrai négatif)

TABLE 6 – Matrice de coûts

où :

- c_f représente le coût fixe d'investigation, appliqué à toute transaction signalée comme suspecte (qu'elle soit réellement frauduleuse ou non).
- A_i désigne le coût variable d'un faux négatif pour la transaction i , correspondant au montant de la transaction frauduleuse non détectée.
- Le coût d'un vrai négatif est nul ($C_i(0|0) = 0$), car une transaction légitime correctement classée ne génère aucun coût supplémentaire.

Étant donné cette matrice de coûts, nous pouvons définir le coût total associé à l'utilisation d'un modèle de classification de score $\pi(\cdot)$ sur un ensemble de données $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$. Pour une transaction i , le coût attendu s'exprime en fonction de la probabilité prédite $\pi_i = \pi(\mathbf{x}_i)$ et des coûts associés :

$$E[\text{Coût}_i] = y_i[\hat{y}_i C_i(1|1) + (1 - \hat{y}_i) C_i(0|1)] + (1 - y_i)[\hat{y}_i C_i(1|0) + (1 - \hat{y}_i) C_i(0|0)]$$

En remplaçant par les valeurs de notre matrice de coûts et en utilisant l'espérance conditionnelle $E[\hat{y}_i|\mathbf{x}_i] = \pi_i$, nous obtenons :

$$E[\text{Coût}_i] = y_i[\pi_i c_f + (1 - \pi_i) A_i] + (1 - y_i)[\pi_i c_f + 0]$$

En développant cette expression :

$$E[\text{Coût}_i] = y_i \pi_i c_f + y_i (1 - \pi_i) A_i + (1 - y_i) \pi_i c_f$$

$$E[\text{Coût}_i] = y_i A_i - y_i \pi_i A_i + \pi_i c_f [y_i + (1 - y_i)]$$

$$E[\text{Coût}_i] = y_i A_i (1 - \pi_i) + \pi_i c_f$$

Le coût attendu moyen (Average Expected Cost, AEC) sur l'ensemble des données est alors défini comme :

$$AEC(\boldsymbol{\beta}) = \frac{1}{N} \sum_{i=1}^N [y_i (1 - \pi_i) A_i + \pi_i c_f]$$

où $\pi_i = \pi(\mathbf{x}_i) = \frac{e^{\mathbf{x}_i^T \boldsymbol{\beta}}}{1 + e^{\mathbf{x}_i^T \boldsymbol{\beta}}}$ représente la probabilité prédite par le modèle de régression logistique.

L'objectif de la régression logistique sensible aux coûts (CSLogit) est de trouver les paramètres $\boldsymbol{\beta}$ qui minimisent le coût attendu moyen :

$$\boldsymbol{\beta}^* = \arg \min_{\boldsymbol{\beta}} AEC(\boldsymbol{\beta})$$

Pour résoudre ce problème d'optimisation, nous devons calculer le gradient de l'AEC par rapport aux paramètres $\boldsymbol{\beta}$. En dérivant l'expression de l'AEC, nous obtenons :

$$\frac{\partial AEC}{\partial \beta} = \frac{1}{N} \sum_{i=1}^N \left[-y_i A_i \frac{\partial \pi_i}{\partial \beta} + c_f \frac{\partial \pi_i}{\partial \beta} \right]$$

$$\frac{\partial AEC}{\partial \beta} = \frac{1}{N} \sum_{i=1}^N (c_f - y_i A_i) \frac{\partial \pi_i}{\partial \beta}$$

Or,

$$\frac{\partial \pi_i}{\partial \beta} = \pi_i(1 - \pi_i) \mathbf{x}_i$$

En substituant cette expression, nous obtenons le gradient :

$$\nabla_{\beta} AEC = \frac{1}{N} \sum_{i=1}^N \pi_i(1 - \pi_i)(c_f - y_i A_i) \mathbf{x}_i$$

Toutes fois en utilisant cette approche nous obtenons des probabilité de fraude très faible pour des coûts d'investigation élevés et des probabilité de fraude très élevés pour des coûts d'investigation faible. Pour palier à ce problème nous avons adopté une approche intermédiaire qui consiste non seulement à minimiser le coût moyen de la fraude mais aussi à maximiser la vraisemblance.

La fonction objectif composite s'écrit :

$$\mathcal{L}(\beta) = \alpha \cdot AEC(\beta) - (1 - \alpha) \cdot \ell(\beta)$$

où $\alpha \in [0, 1]$ est un hyperparamètre qui contrôle la sensibilité aux coûts , et $\ell(\beta)$ représente la log-vraisemblance classique de la régression logistique.

Le Tableau 7 ci-après présente les résultats obtenus par le modèle r différentes valeurs du coût fixe d'investigation c_f :

TABLE 7 – Performances du modèle CSLogit selon le coût fixe d'investigation c_f

c_f	Recall	Precision	AUC-ROC	AUC-PR	Accuracy	G-Mean	Cross-Entropy	Brier Score	F1-Score
5	0,601	0,832	0,950	0,672	0,999	0,775	0,0059	0,00089	0,698
15	0,595	0,889	0,948	0,665	0,999	0,771	0,0052	0,00080	0,713
25	0,723	0,856	0,941	0,697	0,999	0,850	0,0084	0,00070	0,784
35	0,716	0,876	0,938	0,691	0,999	0,846	0,0085	0,00071	0,788
45	0,716	0,869	0,935	0,679	0,999	0,846	0,0087	0,00071	0,785
55	0,716	0,855	0,932	0,670	0,999	0,846	0,0088	0,00070	0,779
65	0,547	0,827	0,913	0,576	0,999	0,740	0,0075	0,00099	0,659

On constate que pour des faibles coût d'investigation , le modèle privilégie la précision au détriment du rappel : avec un c_f de 5 seulement 60% des fraudes sont détectées, mais avec une précision élevée (83-89%). Cependant, bien que les performances du CSLogit demeurent modeste par rapport aux meilleurs modèles obtenus lors des étapes précédentes (notamment en termes d'AUC-ROC et de rappel), elles restent néanmoins satisfaisantes et présentent l'avantage d'intégrer explicitement la structure de coûts dans le processus d'optimisation.

Voici les coûts totaux calculés sur test selon les coûts d'investigation passés en entraînement :

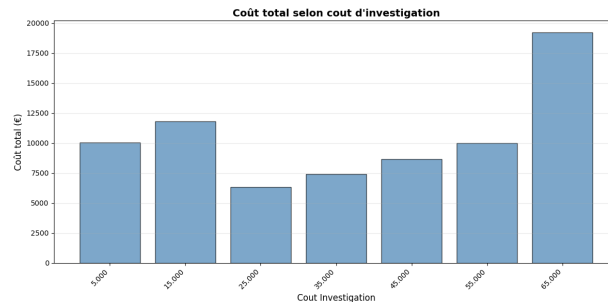


FIGURE 5 – Coût de la fraude selon coût de l'investigation

6 Conclusion

La détection de fraude constitue un défi majeur en raison de son caractère rare et évolutif. Dans ce projet, nous avons mis en œuvre et comparé plusieurs techniques d'apprentissage supervisé et non supervisé, ainsi que diverses méthodes de rééchantillonnage pour traiter le déséquilibre entre les classes.

Les méthodes non supervisées se sont révélées capables de détecter la fraude dans une certaine mesure. L'Isolation Forest a permis d'identifier 68% des fraudes parmi les anomalies détectées, tandis que le HDBSCAN a atteint des performances encore plus élevées soit 84,15% des fraudes sont considérées comme des anomalies.

Les modèles supervisés quant à eux ont obtenu des résultats nettement supérieurs, en particulier le Random Forest et XGBoost. Ces algorithmes, de par leur structure, sont capables de modéliser des relations non linéaires complexes et des interactions entre variables, contrairement à la régression logistique qui reste limitée par sa nature linéaire. Le rééchantillonnage s'est avéré important pour améliorer les performances : en rééquilibrant les classes, ces techniques ont permis aux modèles de mieux apprendre les caractéristiques de la classe minoritaire (les fraudes) et ainsi d'améliorer significativement leur capacité de détection.

Les résultats obtenus pourraient être davantage améliorés par une exploration plus exhaustive de l'espace des hyperparamètres et un réglage plus fin des modèles. De plus, d'autres architectures n'ont pas été testées dans ce travail et pourraient potentiellement offrir des performances supérieures.

Le choix final du modèle résulte d'un arbitrage entre le coût d'investigation et le coût de la fraude, tout en tenant compte des éventuelles obligations réglementaires de détection. Dans un contexte où l'obligation de détection est prioritaire, il conviendrait d'augmenter le recall au détriment de la précision afin de capturer le maximum de transactions frauduleuses, quitte à générer davantage de fausses alertes. Toutefois, dans notre cadre d'analyse, nous avons privilégié une approche de rentabilité cherchant à optimiser l'arbitrage coût-bénéfice.

Au regard de l'ensemble des résultats, nous avons conclu que le modèle XGBoost entraîné sur la base SMOTE 10/90 représente le meilleur compromis, avec un recall de 81%, une précision de 93,75% et un F1-score de 0,870. Ce choix pourrait être davantage affiné si nous disposions d'une estimation précise du coût d'investigation, permettant ainsi d'intégrer explicitement cette information dans le processus de décision. En l'état actuel, ce modèle offre le meilleur équilibre entre performance de détection et fiabilité des alertes, compte tenu des informations dont nous disposons.

7 Bibliographie

Références

- [Banulescu-Radu, 2025] Banulescu-Radu, D. (2025). Fraud detection. Course notes.
- [Batista et al., 2004] Batista, G. E., Prati, R. C., and Monard, M. C. (2004). A study of the behavior of several methods for balancing machine learning training data. *ACM SIGKDD Explorations Newsletter*, 6(1) :20–29.
- [Chawla et al., 2002] Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. (2002). Smote : Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16 :321–357.
- [Gowda and Krishna, 1979] Gowda, K. C. and Krishna, G. (1979). The condensed nearest neighbor rule using the concept of mutual nearest neighborhood. *IEEE Transactions on Information Theory*, 25(4) :488–490.
- [He et al., 2008] He, H., Bai, Y., Garcia, E. A., and Li, S. (2008). Adasyn : Adaptive synthetic sampling approach for imbalanced learning. *IEEE International Joint Conference on Neural Networks*, pages 1322–1328.
- [Tremblay et al., 2022] Tremblay, M. et al. (2022). Extensions de smote pour données mixtes. *Revue d’Intelligence Artificielle*.
- [Yankol Schalck, 2025] Yankol Schalck, M. (2025). Auto insurance fraud detection : Leveraging cost sensitive and insensitive algorithms for comprehensive analysis. *Insurance : Mathematics and Economics*, 122 :44–60.

8 Annexes

TABLE 8 – Tableau comparatif des configurations de LogisticRegression pour la détection de fraude.

Configuration	Recall	Precision	AUC ROC	AUC PR	Accuracy	G Mean	Cross Entropy	Brier Score	F1 Score
Initiale	0.770 270 3	0.826 087 0	0.975 878 2	0.753 066 5	0.999 321 2	0.877 526 9	0.102 215 5	0.021 304 0	0.797 202 8
adasyn_05_95	0.777 027 0	0.839 416 1	0.976 162 4	0.735 771 7	0.999 356 2	0.881 378 3	0.053 741 6	0.009 072 1	0.807 017 5
adasyn_10_90	0.777 027 0	0.839 416 1	0.974 804 6	0.739 954 6	0.999 356 2	0.881 378 3	0.051 039 0	0.008 706 2	0.807 017 5
adasyn_20_80	0.756 756 8	0.868 217 0	0.974 743 6	0.750 667 4	0.999 379 7	0.869 830 8	0.052 340 7	0.009 072 0	0.808 664 3
borderlinesmote_05_95	0.797 297 3	0.761 290 3	0.958 320 6	0.747 364 5	0.999 215 6	0.892 720 7	0.024 340 7	0.003 477 3	0.778 878 0
borderlinesmote_10_90	0.783 783 8	0.811 188 8	0.950 621 4	0.747 717 4	0.999 309 5	0.885 176 0	0.017 840 7	0.002 830 6	0.797 251 4
borderlinesmote_20_80	0.777 027 0	0.809 859 2	0.947 757 2	0.744 361 4	0.999 298 2	0.881 352 4	0.016 500 7	0.002 640 4	0.793 103 4
cnn	0.750 000 0	0.685 185 2	0.970 366 2	0.708 220 3	0.998 970 4	0.865 766 5	0.112 971 4	0.021 002 7	0.716 129 0
nearmiss_05_95	0.432 432 4	0.118 738 3	0.962 753 6	0.084 658 2	0.993 458 1	0.655 761 9	0.422 960 7	0.067 864 0	0.186 317 0
nearmiss_10_90	0.445 945 9	0.150 342 5	0.961 382 9	0.105 684 2	0.994 675 4	0.666 330 4	0.335 570 8	0.067 947 3	0.224 872 1
nearmiss_20_80	0.398 648 6	0.046 862 8	0.945 222 6	0.033 569 5	0.984 913 9	0.626 928 7	0.832 110 3	0.108 863 1	0.083 866 4
randomoversampling_05_95	0.743 243 2	0.859 375 0	0.976 054 2	0.750 944 1	0.999 344 7	0.862 025 3	0.101 740 7	0.020 987 6	0.797 101 4
randomoversampling_10_90	0.736 486 5	0.865 079 4	0.976 663 6	0.744 912 8	0.999 344 7	0.858 102 5	0.102 183 6	0.021 162 7	0.795 620 4
randomoversampling_20_80	0.770 270 3	0.826 087 0	0.976 238 8	0.744 042 1	0.999 321 2	0.877 526 9	0.101 957 6	0.021 142 6	0.797 202 8
randomundersampling_05_95	0.682 432 4	0.677 852 3	0.969 526 9	0.674 007 7	0.998 887 7	0.825 862 1	0.152 017 2	0.028 006 5	0.680 134 7
randomundersampling_10_90	0.736 486 5	0.612 359 6	0.974 871 9	0.648 530 6	0.998 736 0	0.857 841 4	0.117 064 9	0.023 628 4	0.668 711 7
randomundersampling_20_80	0.729 729 7	0.548 223 4	0.970 642 9	0.595 492 5	0.998 490 2	0.853 796 3	0.118 200 1	0.022 155 4	0.626 087 0
smote_05_95	0.777 027 0	0.845 588 2	0.975 049 8	0.734 685 6	0.999 367 9	0.881 378 3	0.050 574 6	0.008 599 3	0.809 859 2
smote_10_90	0.777 027 0	0.839 416 1	0.974 976 5	0.749 007 9	0.999 356 2	0.881 378 3	0.049 746 9	0.008 464 9	0.807 017 5
smote_20_80	0.777 027 0	0.839 416 1	0.974 709 1	0.747 895 9	0.999 356 2	0.881 378 3	0.049 453 3	0.008 403 8	0.807 017 5
smoteenn_05_95	0.763 513 5	0.843 283 6	0.958 420 2	0.744 732 0	0.999 344 7	0.873 685 2	0.029 448 3	0.004 840 4	0.801 418 4
smoteenn_10_90	0.756 756 8	0.854 961 8	0.960 834 2	0.744 535 6	0.999 356 2	0.869 821 2	0.034 959 4	0.005 386 6	0.802 867 1
smoteenn_20_80	0.743 243 2	0.873 015 9	0.960 885 1	0.733 092 1	0.999 367 9	0.862 035 3	0.039 439 7	0.006 288 4	0.802 919 7
svmsmote_05_95	0.777 027 0	0.756 578 9	0.959 566 3	0.742 245 1	0.999 180 7	0.881 300 3	0.021 193 6	0.002 754 2	0.766 666 7
svmsmote_10_90	0.777 027 0	0.787 671 2	0.952 515 4	0.748 559 8	0.999 250 6	0.881 331 3	0.014 803 5	0.002 236 4	0.782 312 9
svmsmote_20_80	0.750 000 0	0.810 219 0	0.948 845 3	0.743 834 4	0.999 262 5	0.865 892 8	0.013 784 2	0.002 102 4	0.778 947 4

TABLE 9 – Tableau comparatif des configurations de RandomForest pour la détection de fraude.

Configuration	Recall	Precision	AUC ROC	AUC PR	Accuracy	G Mean	Cross Entropy	Brier Score	F1 Score
Initiale	0.790 540 5	0.921 259 8	0.959 513 7	0.836 718 4	0.999 520 1	0.889 071 3	0.006 592 4	0.000 471 2	0.850 909 1
adasyn_05_95	0.831 081 1	0.860 139 9	0.966 870 7	0.822 519 3	0.999 473 3	0.911 530 1	0.008 836 4	0.000 705 8	0.845 360 8
adasyn_10_90	0.810 810 8	0.882 352 9	0.957 872 5	0.836 898 3	0.999 485 0	0.900 366 4	0.009 062 3	0.000 782 0	0.845 070 4
adasyn_20_80	0.797 297 3	0.893 939 4	0.981 167 0	0.850 709 7	0.999 485 0	0.892 841 9	0.007 729 4	0.000 891 9	0.842 857 1
borderlinesmote_05_95	0.777 027 0	0.905 511 8	0.976 683 6	0.822 086 4	0.999 473 3	0.881 429 0	0.004 615 9	0.000 570 7	0.836 363 6
borderlinesmote_10_90	0.797 297 3	0.887 218 0	0.979 857 9	0.820 012 1	0.999 473 3	0.892 837 2	0.004 391 6	0.000 585 1	0.839 858 4
borderlinesmote_20_80	0.783 783 8	0.920 634 9	0.956 512 2	0.824 606 4	0.999 508 4	0.885 263 7	0.007 035 3	0.000 506 3	0.846 715 3
cnn	0.783 783 8	0.865 671 6	0.979 629 4	0.752 424 6	0.999 414 8	0.885 222 5	0.036 302 4	0.004 308 8	0.822 695 0
nearmiss_05_95	0.797 297 3	0.825 174 8	0.973 286 8	0.752 726 3	0.999 356 2	0.892 784 4	0.042 173 2	0.007 798 9	0.810 996 6
nearmiss_10_90	0.763 513 5	0.869 230 8	0.972 169 4	0.742 364 9	0.999 391 4	0.873 705 7	0.108 537 6	0.033 093 1	0.812 949 6
nearmiss_20_80	0.797 297 3	0.861 313 9	0.961 118 3	0.720 531 8	0.999 426 5	0.892 815 8	0.213 818 2	0.072 132 4	0.828 070 2
randomoversampling_05_95	0.790 540 5	0.914 062 5	0.965 290 9	0.838 068 9	0.999 508 4	0.889 065 8	0.007 105 2	0.000 552 4	0.847 826 1
randomoversampling_10_90	0.790 540 5	0.921 259 8	0.967 443 9	0.830 201 2	0.999 520 1	0.889 071 3	0.007 161 7	0.000 572 8	0.850 909 1
randomoversampling_20_80	0.804 054 1	0.888 059 7	0.967 999 7	0.834 086 4	0.999 485 0	0.896 611 6	0.007 016 8	0.000 566 6	0.843 971 6
randomundersampling_05_95	0.804 054 1	0.838 028 2	0.971 742 6	0.777 920 5	0.999 391 4	0.896 569 9	0.023 200 9	0.002 721 0	0.820 689 7
randomundersampling_10_90	0.804 054 1	0.782 894 7	0.979 025 1	0.746 662 4	0.999 274 1	0.896 516 6	0.031 607 4	0.003 724 6	0.793 333 3
randomundersampling_20_80	0.770 270 3	0.870 229 0	0.982 100 7	0.728 258 1	0.999 403 1	0.877 563 3	0.048 284 4	0.007 375 9	0.817 204 3
smote_05_95	0.817 567 6	0.876 811 6	0.964 285 1	0.842 948 5	0.999 485 0	0.904 104 3	0.008 192 0	0.000 681 4	0.846 153 8
smote_10_90	0.790 540 5	0.886 363 6	0.977 747 1	0.841 737 4	0.999 461 6	0.889 045 3	0.009 272 3	0.001 046 4	0.835 714 3
smote_20_80	0.804 054 1	0.888 059 7	0.972 445 4	0.842 140 6	0.999 485 0	0.896 611 6	0.008 763 0	0.000 868 0	0.843 971 6
smoteenn_05_95	0.804 054 1	0.901 515 2	0.943 851 1	0.813 384 4	0.999 508 4	0.896 622 4	0.009 114 8	0.000 495 4	0.850 000 0
smoteenn_10_90	0.804 054 1	0.888 059 7	0.961 971 8	0.808 139 0	0.999 485 0	0.896 611 6	0.008 165 5	0.000 576 9	0.843 971 6
smoteenn_20_80	0.797 297 3	0.900 763 4	0.964 547 3	0.834 566 3	0.999 496 7	0.892 847 3	0.009 134 1	0.000 926 3	0.845 878 1
svmsmote_05_95	0.790 540 5	0.921 259 8	0.979 500 3	0.834 966 5	0.999 520 1	0.889 071 3	0.003 435 2	0.000 487 0	0.850 909 1
svmsmote_10_90	0.797 297 3	0.893 939 4	0.984 783 6	0.821 285 3	0.999 485 0	0.892 841 9	0.004 284 5	0.000 577 3	0.842 857 1
svmsmote_20_80	0.777 027 0	0.920 000 0	0.980 100 6	0.822 025 3	0.999 496 7	0.881 439 5	0.004 011 1	0.000 562 3	0.842 490 8

TABLE 10 – Tableau comparatif des configurations de NeuralNet pour la détection de fraude.

Configuration	Recall	Precision	AUC ROC	AUC PR	Accuracy	G Mean	Cross Entropy	Brier Score	F1 Score
Initiale	0.750 000 0	0.932 773 1	0.985 042 9	0.832 121 7	0.999 473 3	0.865 984 8	0.002 866 4	0.000 490 6	0.831 460 7
adasyn_05_95	0.756 756 8	0.903 225 8	0.970 554 2	0.814 649 3	0.999 438 2	0.869 855 6	0.004 923 6	0.000 622 3	0.823 529 4
adasyn_10_90	0.722 973 0	0.877 049 2	0.951 199 4	0.785 199 8	0.999 344 7	0.850 202 7	0.010 062 3	0.000 692 0	0.792 592 6
adasyn_20_80	0.729 729 7	0.878 048 8	0.960 761 6	0.762 603 1	0.999 356 2	0.854 166 6	0.013 477 2	0.000 674 0	0.797 047 9
borderlinesmote_05_95	0.763 513 5	0.957 627 1	0.965 522 2	0.826 383 1	0.999 531 8	0.873 766 6	0.003 637 9	0.000 528 3	0.849 624 1
borderlinesmote_10_90	0.756 756 8	0.933 333 3	0.978 097 9	0.812 085 3	0.999 485 0	0.869 877 2	0.003 413 0	0.000 545 7	0.835 820 9
borderlinesmote_20_80	0.736 486 5	0.900 826 4	0.958 468 6	0.802 415 3	0.999 403 1	0.858 127 7	0.004 453 3	0.000 598 3	0.810 408 9
cnn	0.729 729 7	0.843 750 0	0.975 430 7	0.757 342 7	0.999 298 2	0.854 142 3	0.035 538 7	0.007 150 5	0.782 608 7
nearmiss_05_95	0.770 270 3	0.844 444 4	0.965 524 0	0.717 713 4	0.999 356 2	0.877 542 0	0.133 764 7	0.034 902 9	0.805 654 5
nearmiss_10_90	0.202 702 7	0.162 162 2	0.942 583 0	0.084 158 1	0.996 804 7	0.449 816 5	0.487 493 6	0.061 484 2	0.180 180 2
nearmiss_20_80	0.668 918 9	0.026 281 1	0.930 256 8	0.017 784 4	0.956 496 6	0.800 096 4	0.435 362 3	0.091 054 0	0.050 574 7
randomoversampling_05_95	0.702 702 7	0.936 936 9	0.954 383 4	0.767 671 2	0.999 403 1	0.838 239 1	0.006 345 7	0.001 189 3	0.803 088 8
randomoversampling_10_90	0.695 945 9	0.944 954 1	0.957 872 9	0.785 724 7	0.999 403 1	0.834 204 2	0.006 842 7	0.000 697 7	0.801 556 4
randomoversampling_20_80	0.736 486 5	0.947 826 1	0.963 654 0	0.793 999 7	0.999 473 3	0.858 158 4	0.008 486 6	0.000 667 7	0.828 897 3
randomundersampling_05_95	0.790 540 5	0.841 726 6	0.975 427 6	0.748 894 0	0.999 380 4	0.889 008 8	0.013 526 7	0.001 674 6	0.815 331 0
randomundersampling_10_90	0.777 027 0	0.851 851 9	0.979 085 3	0.782 033 8	0.999 380 4	0.881 388 4	0.019 158 7	0.003 528 6	0.812 720 8
randomundersampling_20_80	0.743 243 2	0.769 230 8	0.975 693 6	0.719 732 9	0.999 169 0	0.861 949 2	0.040 378 7	0.008 394 7	0.756 013 7
smote_05_95	0.783 783 8	0.811 188 8	0.950 166 9	0.789 752 3	0.999 309 5	0.885 176 0	0.007 300 2	0.000 707 7	0.797 251 4
smote_10_90	0.722 973 0	0.930 434 8	0.950 319 9	0.796 112 3	0.999 426 5	0.850 238 4	0.009 179 8	0.000 614 5	0.813 688 3
smote_20_80	0.743 243 2	0.852 713 2	0.948 072 7	0.771 017 1	0.999 333 0	0.862 020 4	0.007 611 3	0.000 657 2	0.794 223 8
smoteenn_05_95	0.750 000 0	0.917 355 4	0.940 517 5	0.784 487 4	0.999 449 9	0.865 974 8	0.006 633 0	0.000 570 3	0.825 278 8
smoteenn_10_90	0.770 270 3	0.904 761 9	0.967 111 1	0.800 463 0	0.999 461 6	0.877 589 3	0.005 682 6	0.000 673 2	0.832 116 8
smoteenn_20_80	0.750 000 0	0.932 773 1	0.980 598 6	0.823 338 3	0.999 473 3	0.865 984 8	0.004 402 4	0.000 701 7	0.831 460 7
svmsmote_05_95	0.804 054 1	0.782 894 7	0.950 222 3	0.803 445 1	0.999 274 1	0.896 516 6	0.005 819 8	0.000 691 3	0.793 333 3
svmsmote_10_90	0.770 270 3	0.870 229 0	0.956 961 2	0.793 802 0	0.999 403 1	0.877 563 3	0.003 985 9	0.000 623 3	0.817 204 3
svmsmote_20_80	0.770 270 3	0.863 636 4	0.954 807 8	0.788 649 8	0.999 391 4	0.877 558 0	0.004 214 9	0.000 657 4	0.814 285 7

TABLE 11 – Tableau comparatif des configurations de AdaBoost pour la détection de fraude.

Configuration	Recall	Precision	AUC ROC	AUC PR	Accuracy	G Mean	Cross Entropy	Brier Score	F1 Score
Initiale	0.797 297 3	0.797 297 3	0.970 957 8	0.713 417 4	0.999 298 2	0.892 758 0	0.169 694 9	0.028 006 4	0.797 297 3
adasyn_05_95	0.783 783 8	0.800 000 0	0.974 871 1	0.725 834 9	0.999 285 8	0.885 165 5	0.201 734 8	0.038 514 3	0.791 808 9
adasyn_10_90	0.716 216 2	0.868 852 5	0.975 300 6	0.730 204 1	0.999 321 2	0.846 215 7	0.200 284 9	0.037 751 9	0.785 185 2
adasyn_20_80	0.770 270 3	0.832 116 8	0.974 718 3	0.727 944 8	0.999 333 0	0.877 532 4	0.208 438 4	0.040 557 6	0.800 000 0
borderlinesmote_05_95	0.756 756 8	0.848 484 8	0.960 423 2	0.711 280 3	0.999 344 7	0.869 816 3	0.158 169 5	0.023 769 7	0.800 000 0
borderlinesmote_10_90	0.790 540 5	0.769 736 8	0.962 607 2	0.712 906 2	0.999 227 5	0.888 941 3	0.159 749 1	0.023 828 7	0.780 000 0
borderlinesmote_20_80	0.763 513 5	0.801 418 4	0.954 524 3	0.711 094 3	0.999 262 5	0.873 648 6	0.200 349 7	0.035 493 2	0.782 006 9
cnn	0.770 270 3	0.863 636 4	0.970 382 7	0.724 248 4	0.999 391 4	0.877 558 3	0.183 231 0	0.032 430 4	0.814 285 7
nearmiss_05_95	0.770 270 3	0.832 116 8	0.969 368 3	0.711 122 0	0.999 333 0	0.877 532 4	0.171 043 4	0.028 419 3	0.800 000 0
nearmiss_10_90	0.695 945 9	0.851 239 7	0.971 405 6	0.689 691 2	0.999 262 5	0.834 145 6	0.184 963 4	0.032 444 6	0.765 799 3
nearmiss_20_80	0.716 216 2	0.854 838 7	0.970 581 4	0.713 802 7	0.999 298 2	0.846 206 4	0.181 015 9	0.031 496 3	0.779 411 8
randomoversampling_05_95	0.797 297 3	0.797 297 3	0.968 969 4	0.717 834 8	0.999 298 2	0.892 758 0	0.173 047 2	0.028 900 4	0.797 297 3
randomoversampling_10_90	0.777 027 0	0.798 611 1	0.972 747 2	0.722 786 1	0.999 274 1	0.881 342 4	0.195 318 9	0.035 542 9	0.787 671 2
randomoversampling_20_80	0.763 513 5	0.837 037 0	0.973 345 7	0.720 415 7	0.999 333 0	0.873 679 9	0.206 632 0	0.039 005 3	0.798 587 6
randomundersampling_05_95	0.668 918 9	0.860 869 6	0.972 554 7	0.695 902 6	0.999 239 0	0.817 798 5	0.174 557 3	0.029 378 4	0.752 851 7
randomundersampling_10_90	0.777 027 0	0.761 589 4	0.972 898 8	0.706 618 6	0.999 192 4	0.881 305 2	0.201 751 5	0.037 683 4	0.769 230 8
randomundersampling_20_80	0.662 162 2	0.765 625 0	0.966 012 3	0.591 387 4	0.999 064 2	0.813 589 6	0.239 708 9	0.051 118 2	0.710 144 9
smote_05_95	0.777 027 0	0.839 416 1	0.974 141 4	0.731 806 3	0.999 356 2	0.881 378 3	0.192 907 6	0.035 138 5	0.807 017 5
smote_10_90	0.756 756 8	0.794 326 2	0.975 079 0	0.722 830 6	0.999 239 0	0.869 770 1	0.191 329 5	0.035 055 0	0.775 086 5
smote_20_80	0.783 783 8	0.828 571 4	0.974 673 4	0.730 657 4	0.999 344 7	0.885 190 6	0.209 672 3	0.040 636 6	0.805 555 6
smoteenn_05_95	0.763 513 5	0.856 060 6	0.967 041 0	0.724 167 5	0.999 367 9	0.873 695 0	0.172 575 2	0.027 908 2	0.807 142 9
smoteenn_10_90	0.804 054 1	0.793 333 3	0.971 820 8	0.729 426 6	0.999 298 2	0.896 528 3	0.175 857 7	0.029 363 6	0.798 657 7
smoteenn_20_80	0.797 297 3	0.830 985 9	0.974 871 1	0.732 688 2	0.999 367 9	0.892 789 2	0.201 583 2	0.037 342 4	0.813 793 1
svmsmote_05_95	0.750 000 0	0.860 465 1	0.962 774 6	0.703 699 4	0.999 356 2	0.865 933 5	0.157 102 4	0.023 165 9	0.801 444 0
svmsmote_10_90	0.756 756 8	0.848 484 8	0.964 394 2	0.719 500 3	0.999 344 7	0.869 816 3	0.155 978 3	0.022 633 0	0.800 000 0
svmsmote_20_80	0.763 513 5	0.849 624 1	0.969 939 8	0.709 603 8	0.999 356 2	0.873 689 9	0.174 327 9	0.027 338 2	0.804 270 4

TABLE 12 – Tableau comparatif des configurations de XGBoost pour la détection de fraude.

Configuration	Recall	Precision	AUC ROC	AUC PR	Accuracy	G Mean	Cross Entropy	Brier Score	F1 Score
Initiale	0.790 540 5	0.921 259 8	0.977 409 7	0.812 555 5	0.999 520 1	0.889 071 3	0.007 439 5	0.001 169 6	0.850 909 1
adasyn_05_95	0.817 567 6	0.883 211 7	0.972 708 9	0.845 816 5	0.999 496 7	0.904 109 5	0.003 712 4	0.000 553 4	0.849 122 8
adasyn_10_90	0.804 054 1	0.915 384 6	0.968 612 0	0.848 329 0	0.999 531 8	0.896 632 7	0.003 871 0	0.000 515 4	0.856 115 1
adasyn_20_80	0.817 567 6	0.902 985 1	0.969 302 1	0.850 165 5	0.999 531 8	0.904 125 8	0.004 121 0	0.000 506 6	0.858 156 0
borderlinesmote_05_95	0.797 297 3	0.936 507 9	0.978 656 2	0.845 071 9	0.999 555 2	0.892 873 1	0.003 465 1	0.000 442 4	0.861 313 9
borderlinesmote_10_90	0.804 054 1	0.908 396 9	0.971 012 4	0.839 568 8	0.999 520 1	0.896 627 7	0.003 383 4	0.000 486 7	0.853 046 5
borderlinesmote_20_80	0.804 054 1	0.908 396 9	0.971 707 6	0.831 405 4	0.999 520 1	0.896 627 7	0.003 439 5	0.000 458 4	0.853 046 5
cnn	0.804 054 1	0.826 388 9	0.982 078 2	0.776 523 9	0.999 367 9	0.896 559 4	0.019 994 4	0.004 822 0	0.815 068 5
nearmiss_05_95	0.621 621 6	0.828 829 0	0.969 954 7	0.671 891 6	0.999 122 1	0.788 342 3	0.240 069 5	0.064 981 7	0.710 424 7
nearmiss_10_90	0.574 324 3	0.876 288 7	0.968 520 4	0.667 320 8	0.999 122 1	0.757 788 9	0.309 352 3	0.079 389 2	0.693 877 6
nearmiss_20_80	0.675 675 7	0.793 650 8	0.967 521 4	0.681 085 8	0.999 133 8	0.821 870 4	0.387 069 3	0.103 933 0	0.729 927 0
randomoversampling_05_95	0.804 054 1	0.922 480 6	0.976 620 2	0.839 705 1	0.999 543 5	0.896 638 4	0.003 921 7	0.000 453 3	0.859 206 5
randomoversampling_10_90	0.797 297 3	0.929 133 9	0.974 958 2	0.838 215 9	0.999 543 5	0.892 868 0	0.003 711 0	0.000 456 6	0.858 181 8
randomoversampling_20_80	0.810 810 8	0.909 090 9	0.975 487 8	0.836 619 7	0.999 531 8	0.900 386 7	0.003 665 3	0.000 455 4	0.857 142 9
randomundersampling_05_95	0.763 513 5	0.896 825 4	0.976 050 6	0.811 460 4	0.999 438 2	0.873 726 1	0.008 091 0	0.001 590 5	0.824 817 5
randomundersampling_10_90	0.783 783 8	0.828 571 4	0.981 262 4	0.814 717 4	0.999 344 7	0.885 190 6	0.013 724 9	0.003 227 6	0.805 555 6
randomundersampling_20_80	0.729 729 7	0.900 000 0	0.980 040 3	0.745 186 9	0.999 391 4	0.854 182 2	0.033 242 8	0.008 511 0	0.805 970 1
smote_05_95	0.824 324 3	0.903 703 7	0.969 877 1	0.845 036 5	0.999 543 5	0.907 853 9	0.003 951 6	0.000 504 1	0.862 190 8
smote_10_90	0.810 810 8	0.937 500 0	0.967 424 4	0.852 445 4	0.999 578 6	0.900 407 8	0.003 812 2	0.000 498 8	0.869 565 2
smote_20_80	0.790 540 5	0.943 548 4	0.968 778 3	0.851 519 6	0.999 555 2	0.889 087 2	0.003 698 4	0.000 574 2	0.860 294 1
smoteenn_05_95	0.810 810 8	0.909 090 9	0.961 564 0	0.842 396 5	0.999 531 8	0.900 386 7	0.003 998 2	0.000 474 6	0.857 142 9
smoteenn_10_90	0.790 540 5	0.921 259 8	0.957 542 0	0.837 851 1	0.999 520 1	0.889 071 3	0.004 033 1	0.000 556 4	0.850 909 1
smoteenn_20_80	0.777 027 0	0.934 959 3	0.975 450 6	0.845 387 5	0.999 520 1	0.881 449 5	0.004 035 6	0.000 550 4	0.848 708 5
svmsmote_05_95	0.804 054 1	0.922 480 6	0.979 863 5	0.833 831 3	0.999 543 5	0.896 638 4	0.003 329 5	0.000 456 1	0.859 206 5
svmsmote_10_90	0.804 054 1	0.937 007 9	0.966 899 0	0.842 950 0	0.999 566 9	0.896 648 6	0.003 255 8	0.000 438 2	0.865 455 0
svmsmote_20_80	0.797 297 3	0.936 507 9	0.976 248 8	0.835 991 6	0.999 555 2	0.892 873 1	0.003 516 5	0.000 437 1	0.861 313 9

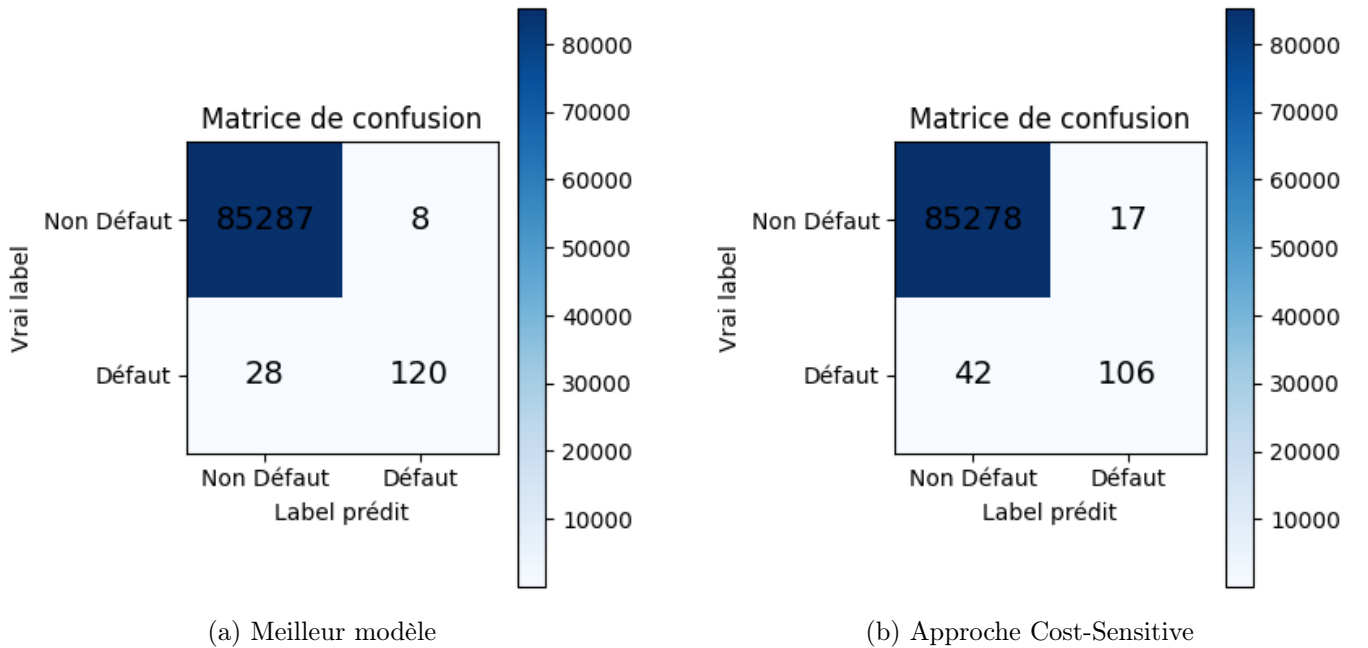
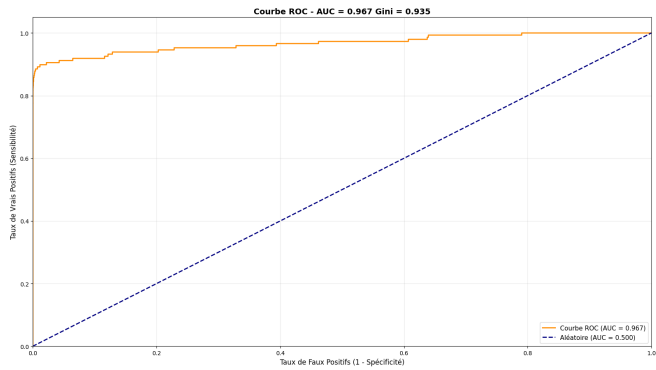
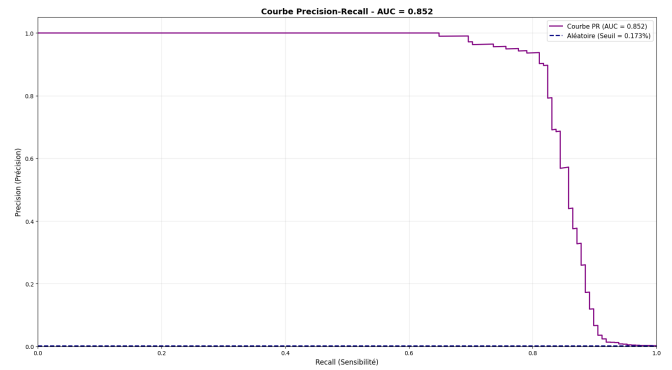


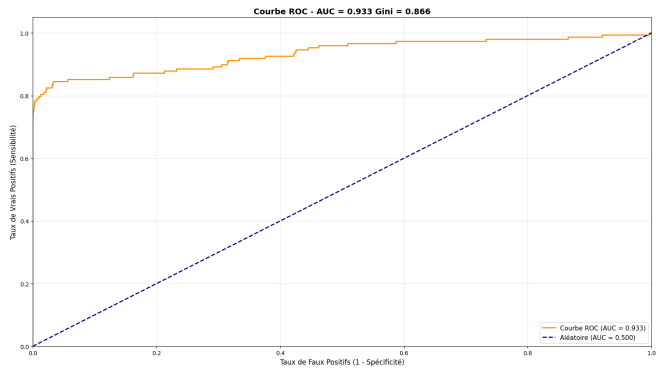
FIGURE 6 – Matrices de Confusion



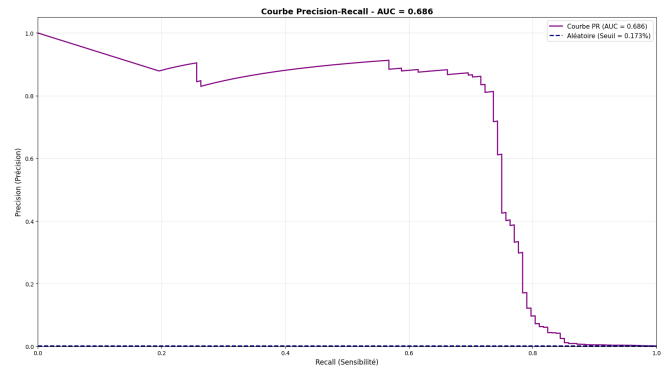
(a) Courbe ROC - Meilleur modèle



(b) Courbe Precision-Recall - Meilleur modèle

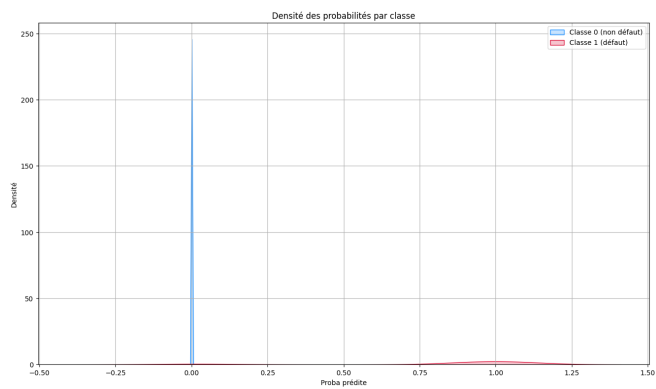


(c) Courbe ROC - Cost-Sensitive

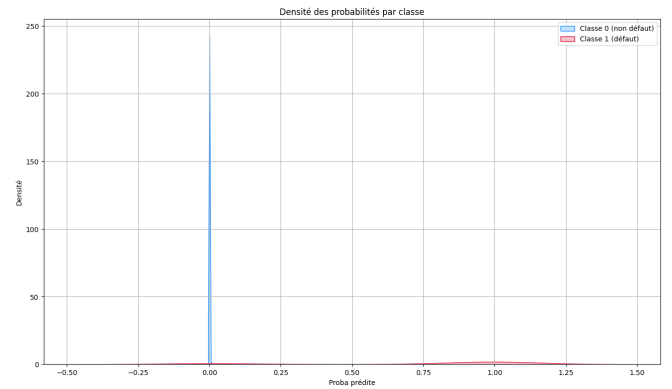


(d) Courbe Precision-Recall - Cost-Sensitive

FIGURE 7 – Comparaison des courbes ROC et Precision-Recall.



(a) Meilleur modèle



(b) Approche Cost-Sensitive

FIGURE 8 – Densité des probabilités prédites par classe.

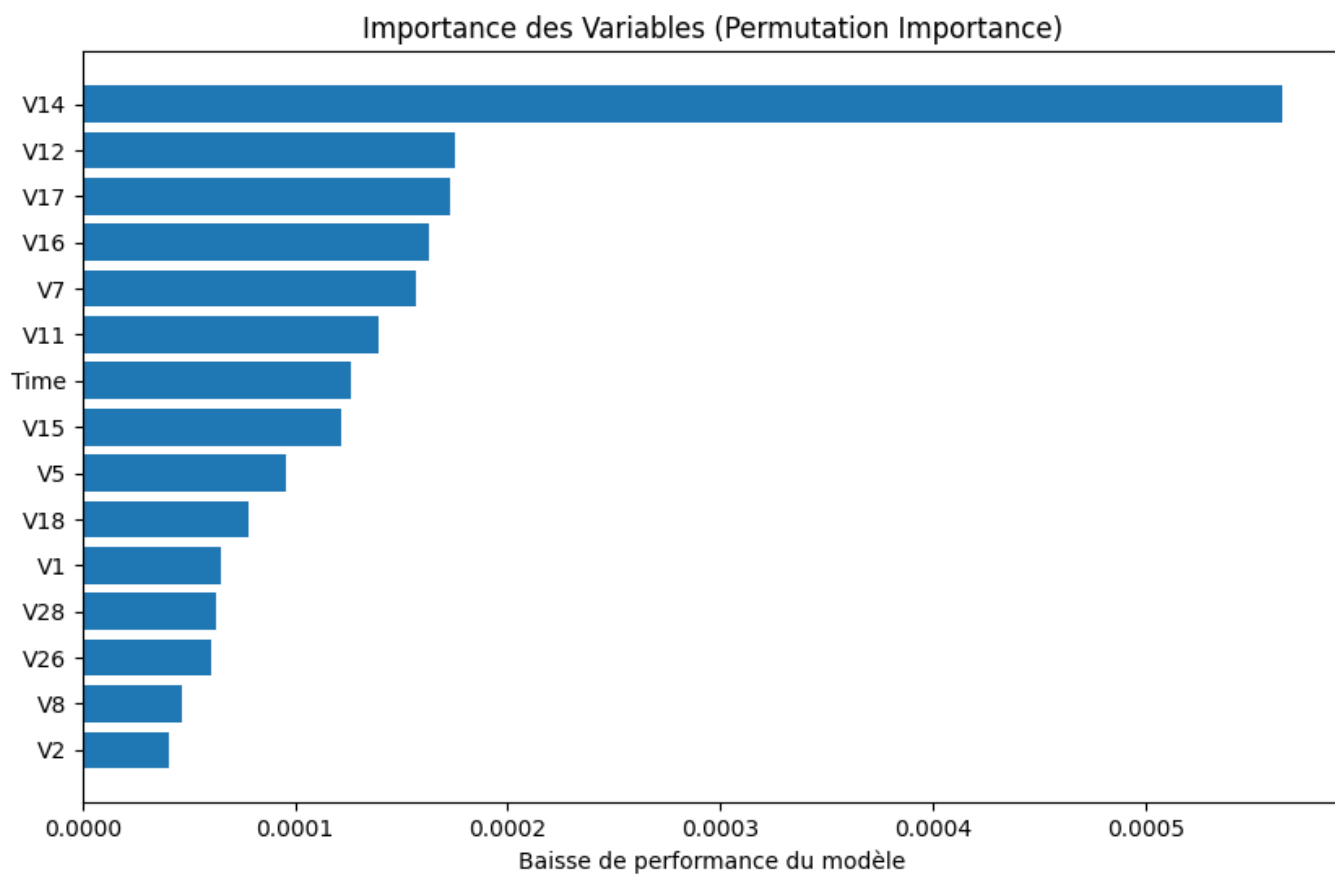


FIGURE 9 – Variables importantes XGBoost SMOTE 10/90